HACKATHON MASTER PLAN

ORBIT

Self-Improving AI Research System for Space Science

Six AI agents that read the literature, propose hypotheses, challenge each other, and run real experiments on NASA exoplanet data - keeping only the improvements that survive an independent evaluation gate. This document is the complete build plan: architecture, agents, ML pipeline, tech stack, 3-day sprint schedule, and the handover system.

Team of 4 · First mission: exoplanet detection

React · FastAPI · PyTorch · MiroFish-inspired multi-agent core

3-DAY SPRINT EDITION · 2026

Table of Contents

Executive Summary and the Winning Angle 1

System Architecture 2

Weekend Pragmatism Decisions . . . 4

The Agent Team: MiroFish-Inspired Core 5

Agent Specifications and Prompt Sketches . . . 6

ML Pipeline: Exoplanet Intelligence 8

Preprocessing . . . 8

Baseline and CNN Champions . . . 8

Metrics, Leakage and the Promotion Gate . . . 9

Backend Engineering (FastAPI) 9

Data Model and the Registry . . . 10

Continuous Integration with GitHub Actions . . . 11

Frontend: React Mission Control 12

Component and State Plan . . . 12

Tech Stack and Justification 13

The Provider-Agnostic LLM Layer . . . 14

Repository Layout and the Handover System 14

The AGENTS.md Protocol . . . 15

PROGRESS.md and HANDOVER.md . . . 15

The 3-Day Sprint Plan 15

Day 0 - The Night Before (2-3 hours) . . . 15

Day 1 - Walking Skeleton (hours 0-10) . . . 16

Day 2 - Feature Complete (hours 10-20) . . . 17

Day 3 - Demo Hardening (hours 20-30) . . . 17

Team Work-Split and Handover Protocol 18

The Handover Ritual . . . 18

Demo Script and Pitch 19

Backup Layers and Pitch Notes . . . 19

Risk Register and Fallbacks 20

Executive Summary and the Winning Angle

ORBIT is a self-improving AI research system for space science, built by a team of four in one weekend. The system answers a research question the way a small research lab would: six specialized AI agents read the literature, propose hypotheses, challenge each other's reasoning, run real experiments on real NASA data, and keep only the improvements that survive an independent evaluation. The first mission is exoplanet detection: given Kepler and TESS light curves, ORBIT's agents try to beat the system's own best detection model, and every attempt - success or failure - is recorded in a reproducible experiment registry that anyone can audit.

72 h	6	2	100%
planned build window	cooperating agents	model families compared	open NASA data

Most hackathon AI-scientist demos are a chat window in front of a scripted flow. ORBIT's pitch is the opposite claim: the system is not scripted, it is governed by evidence. The live demo shows a genuine closed loop - the Hypothesizer proposes a model change, the Skeptic attacks it, the Experimenter trains it on real Kepler data, and the Evaluator either promotes the challenger to champion or records the failure so the system never repeats it. That evaluate-and-keep loop is the novelty of the project: software that gets measurably better at its own research task while the judges watch it happen.

The design borrows its social machinery from MiroFish, an open-source swarm-intelligence engine (81,000+ GitHub stars) that spawns agents with persistent personas and shared memory over a knowledge graph, then harvests their interaction into a prediction report. ORBIT adapts three of its pillars - persona persistence, knowledge-graph memory, and a synthesis step - into a science-grounded arena where agents argue about light curves and model metrics instead of social media sentiment, and where every claim must survive a benchmark gate before it becomes the new system behavior.

Novelty claim	Why it lands with judges
Self-improvement with receipts	Champion vs challenger metrics are stored, versioned and replayable - improvement is measurable, not narrated
Adversarial agent debate	A Skeptic agent with veto power kills weak ideas before compute is spent, mirroring real peer review
Negative results as assets	Failed experiments are logged with a diagnosis, so the system provably avoids repeating mistakes
Real science, real data	NASA Exoplanet Archive labels and MAST light curves; outputs match how candidates are actually vetted

Table 1: The four novelty claims ORBIT defends, and the evidence behind each.

Scope discipline keeps the weekend honest. ORBIT assists research; it does not claim discoveries. Scientific output is flagged as candidate-level, always gated behind the Evaluator's benchmark, and accompanied by a visible disclaimer that human validation is required. This honesty is itself a scoring asset: it demonstrates that the team understands the difference between an impressive demo and a trustworthy scientific instrument, which is exactly the maturity innovation-focused judges reward.

System Architecture

ORBIT is a four-layer architecture: a React mission-control frontend, a FastAPI backend with a WebSocket event bus, an intelligence layer containing the agent orchestrator, the LLM connector and the ML sandbox, and a memory layer holding SQLite, a lightweight knowledge graph and an artifact store. Each layer talks only to its immediate neighbors, which is what allows four teammates to build in parallel against thin, stable interfaces. The entire stack runs on a single laptop with zero external services except the LLM API and NASA data downloads.

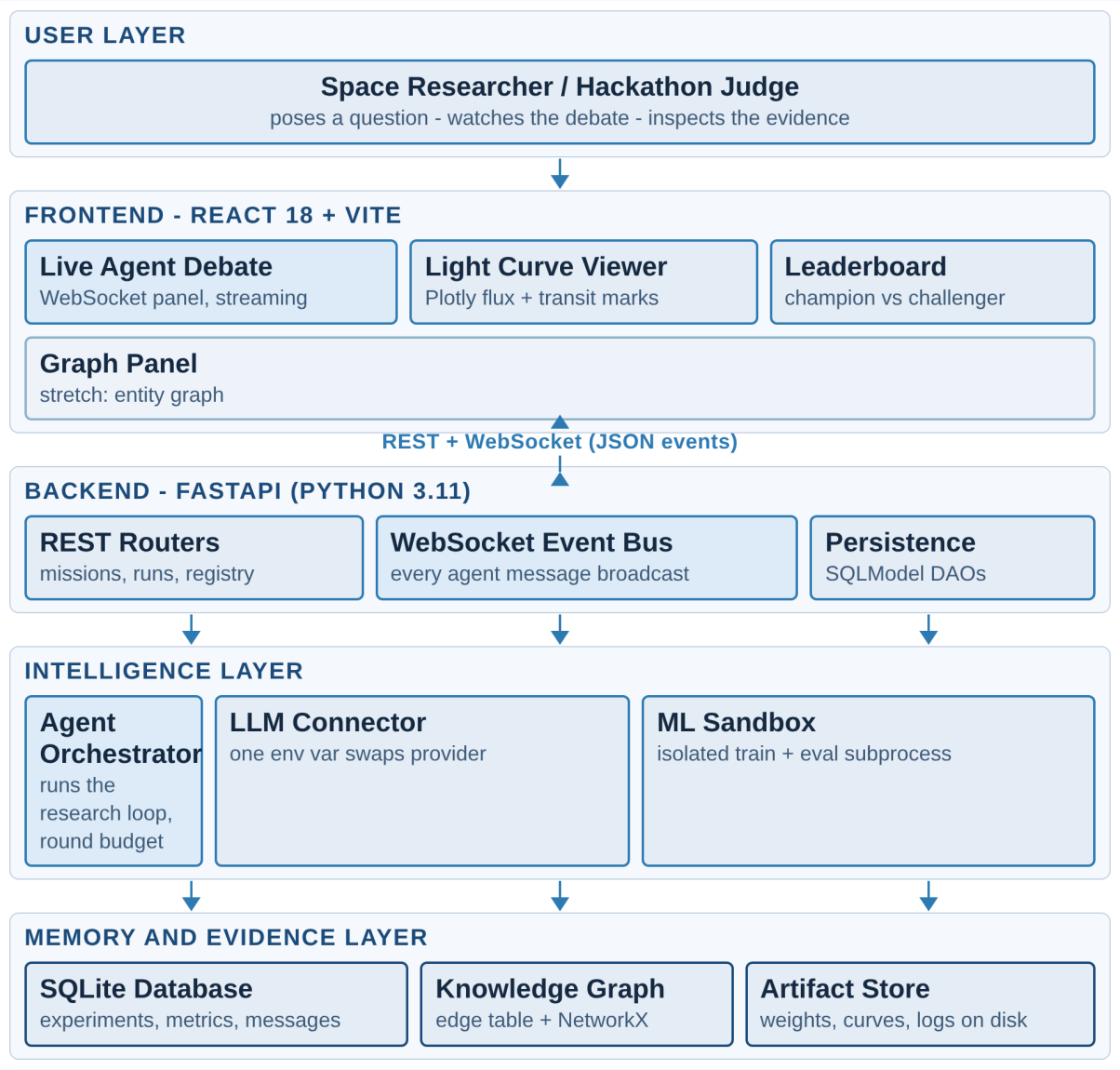


Figure 1: ORBIT system architecture - four layers, thin interfaces, one laptop.

Layer	Component	Responsibility	Owner
Frontend	Mission Control UI	Live debate panel, light curve viewer, leaderboard	Member D
Backend	FastAPI app	REST + WebSocket, persistence, registry API	Member B
Intelligence	Agent Orchestrator	Runs the research loop, enforces the round budget	Member A
Intelligence	LLM Connector	Provider-agnostic chat calls with JSON mode	Member A
Intelligence	ML Sandbox	Isolated subprocess that trains and evaluates models	Member C
Memory	SQLite + Graph + Artifacts	Experiments, metrics, messages, entities, weights	Member B

Table 2: Component responsibilities and the single owner of each.

A mission starts when the user submits a research question. The Orchestrator walks the debate loop shown in Figure 2, and every agent message is broadcast over the WebSocket event bus the moment it is produced, which is what makes the frontend feel alive rather than polled. When the loop reaches the Experimenter, the backend spawns the ML sandbox as a subprocess with a JSON contract on stdin and stdout, so a crashing experiment can never take the API down with it. Results flow back as metric artifacts, the Evaluator compares them against the reigning champion, and the verdict is appended to the experiment registry and pushed to the leaderboard in the same second.

Weekend Pragmatism Decisions

Every architectural choice below optimizes for one metric: the probability of a working demo in 72 hours with four people. The deferred column names the production-grade alternative and, implicitly, the trigger for switching to it later - none of these deferrals are permanent design debts, they are deliberate scope cuts with a documented upgrade path.

Decision	We ship	We defer (and why)
Database	SQLite via SQLModel	Postgres - zero config, file-portable, ample for under 100k rows
Knowledge graph	Edge table + NetworkX in-process	Neo4j - no second server to babysit; the graph fits in memory at our scale
Task queue	BackgroundTasks + subprocess	Celery + Redis - no broker to install at 2 a.m.
Packaging	uvicorn + vite dev servers	Docker Compose - nothing to debug through a container wall
Auth	None (LAN demo)	OAuth - judges are not an attacker model

Table 3: Pragmatic choices versus their production-grade alternatives.

The Agent Team: MiroFish-Inspired Core

MiroFish demonstrates that believable multi-agent behavior comes from three ingredients: persistent personas, shared memory anchored in a knowledge graph, and a synthesis step that turns interaction into a structured report. ORBIT keeps all three but changes the arena: instead of simulated social media posts, our agents produce structured research artifacts - evidence citations, hypotheses, critiques, training runs and verdicts. Each agent is a thin Python class with a role prompt, a tool set and a JSON output contract; the personality lives in the prompt, and the memory lives in the database.

Agent	Core question	Reads	Writes	Temp
Orchestrator	What happens next?	registry, graph	mission plan, round budget	0.2
Researcher	What is known?	papers, prior runs	evidence cards with citations	0.3
Hypothesizer	What should we try?	evidence cards	experiment proposals (JSON config)	0.7
Skeptic	Why will this fail?	proposals, data cards	critiques; veto or pass	0.4
Experimenter	What happens if we test?	approved proposal	run record: metrics + artifacts	0.1
Evaluator	Did it actually improve?	champion + run record	verdict: keep or reject, rationale	0.1

Table 4: The six-agent roster with information flow and sampling temperature.

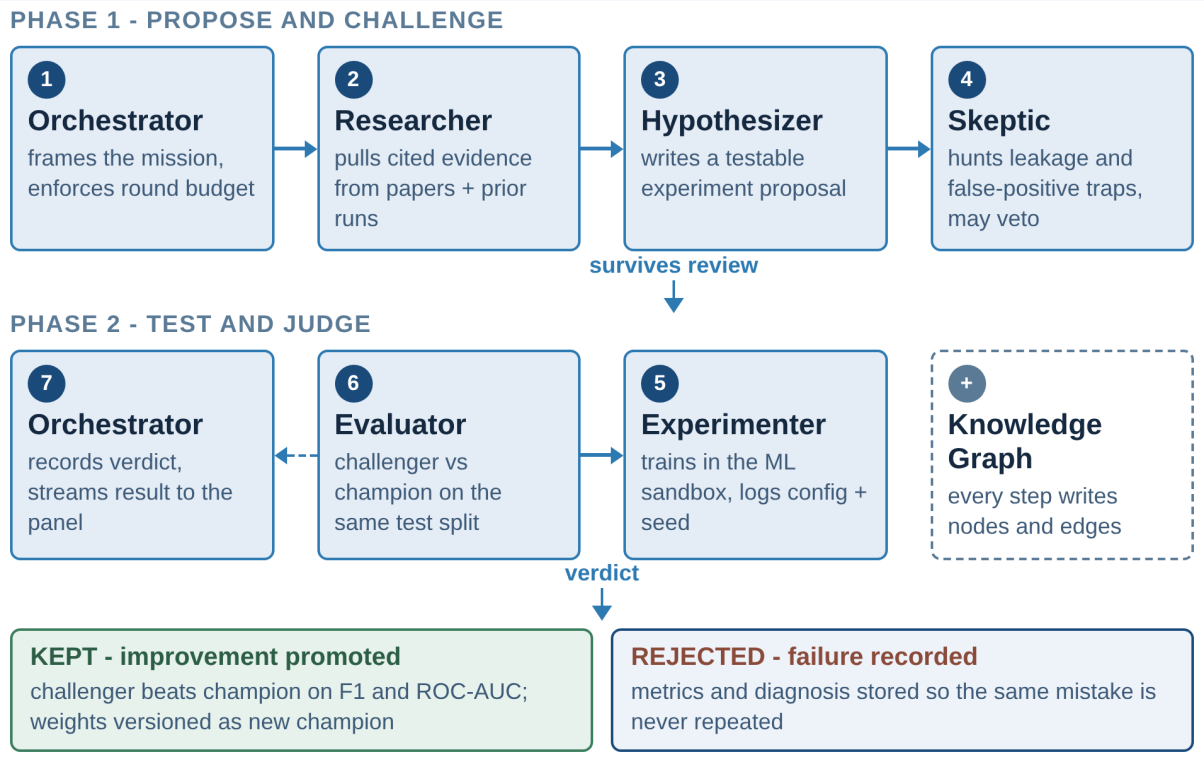


Figure 2: The ORBIT research loop - propose, challenge, test, judge; every step writes to the knowledge graph.

Agent Specifications and Prompt Sketches

Orchestrator

The Orchestrator owns the loop's control state machine: it frames the mission, picks the next agent, and enforces the budget of two revision rounds per mission. Its system prompt is short and mechanical, because creativity here would be a bug rather than a feature. It also serializes agent output into the WebSocket event stream, tagging each message with the agent name, round number and timestamp so the frontend can render the debate as a timeline.

Researcher

The Researcher is the literature-and-memory agent. Its tools are two queries: a keyword search over a compact corpus of exoplanet papers shipped as JSON abstracts, and a graph query for prior experiments similar to the current mission. It must answer with evidence cards, each carrying a source pointer, a one-paragraph summary and a relevance score. This structure forces citations into the debate from the first message and gives the Skeptic something concrete to attack.

Hypothesizer

The Hypothesizer converts evidence into a falsifiable proposal: a complete experiment config as JSON - model family, hyperparameters, feature set, expected metric gain and the reasoning behind them. Its temperature is the highest in the team (0.7) because idea diversity is its job. One prompt rule matters disproportionately: the proposal must name the single change relative to the current champion, for example adding two detrended features while keeping the architecture. Single-change proposals keep experiments interpretable and failures diagnosable.

Skeptic

The Skeptic reviews every proposal before compute is spent, with an explicit veto. Its prompt lists the classic failure modes of exoplanet machine learning - test-set leakage across stars, folded-light-curve artifacts, class imbalance, overfitting to injection noise - and asks it to check the proposal against each one. A veto must include a reason and, wherever possible, a fix. In the demo this agent produces the most memorable moments: watching one AI argue against another AI's plan is the swarm-intelligence hook that judges remember hours later.

Experimenter

The Experimenter is deliberately the least creative agent: it executes exactly what was approved, with no improvisation. It renders the proposal config into a command for the ML sandbox subprocess, streams progress logs back to the panel, and records the run - config hash, git commit, random seed, dataset fingerprint, metrics and artifact paths. Reproducibility is its only key performance indicator; a run that cannot be replayed from its record is marked invalid and excluded from promotion.

Evaluator

The Evaluator applies the promotion gate: the challenger must beat the champion on F1 and ROC-AUC on the same frozen test split, by at least the stated margin, to be kept. It writes a verdict card containing the numbers, a plain-language rationale and - on rejection - a diagnosis of the likely failure cause. Verdicts are first-class database rows rather than chat lines, which is what turns the sentence the system improved from a claim into a query.

Agent messages are JSON objects with five fields: agent, round, type, payload and refs. The type field is limited to evidence_card, proposal, critique, run_record, verdict and status. This schema doubles as the frontend rendering contract and as a continuous-integration target: a unit test feeds canned transcripts through the orchestrator and asserts schema compliance, so a prompt edit that breaks the protocol fails the pipeline before it can fail the demo.

ML Pipeline: Exoplanet Intelligence

The ML layer gives the agent debate something real to argue about. NASA's ExoMiner has already proved that deep networks can validate exoplanets from Kepler photometry at superhuman precision, and the Exoplanet Archive's KOI table provides tens of thousands of labeled candidates and false positives. ORBIT does not need to match ExoMiner; it needs two honest models from different families - a classical baseline and a 1D CNN - so the agents have a meaningful improvement loop to run. Both models train on the same frozen splits and are scored by the same evaluation harness, which makes champion-versus-challenger comparisons apples to apples.

Item	Plan
Labels	NASA Exoplanet Archive KOI cumulative table: CONFIRMED + CANDIDATE versus FALSE POSITIVE
Signals	MAST light curves - Kepler quarters first; TESS sectors where download is fast
Volume	1,500-2,000 targets cached locally on Day 1 (bounded by download time, not by design)
Splits	Grouped by host star: 70 / 15 / 15 train / validation / test, frozen to disk with a fingerprint
Unit	One folded, binned flux vector of 400 points per target plus 12 engineered features

Table 5: Data plan - sources, volume, splits and the learning unit.

Preprocessing

Preprocessing is where exoplanet machine learning is won, so the pipeline is deliberately boring and deterministic. The steps are: download long-cadence PDCSAP flux with lightkurve; remove NaNs and clip sigma outliers; detrend with a Savitzky-Golay filter; normalize; fold at the KOI table's period and epoch; bin into a fixed 400-point vector; and store the result as a NumPy archive alongside the twelve engineered features - transit depth, duration, signal-to-noise, orbital period, radius ratio and companion counts among them. Every step is a pure function with a cached output, so re-runs are instant and the Evaluator can trust that challenger and champion saw byte-identical inputs.

Baseline and CNN Champions

Model	Family	Input	Key details
orbit-baseline-v1	Gradient boosting (sklearn)	12 engineered features	300 trees, depth 6; trains in seconds; a hard bar to clear on tabular features
orbit-cnn-v1	1D CNN (PyTorch)	400-bin folded flux	Conv1D(32) - MaxPool - Conv1D(64) - MaxPool - Conv1D(96) - GlobalPool - Dropout 0.3 - FC(64) - sigmoid; about 90k parameters

Table 6: The two model families the agents will improve upon.

The expected outcome, based on published results, is a baseline near 0.85-0.90 F1 with the CNN at or slightly above it after a few minutes of CPU training. That is exactly the regime the demo wants: close enough that the debate is genuine, separated enough that a smart feature or augmentation can demonstrably win. The agents' first missions therefore target interpretable improvements - class weighting, depth-feature engineering, period-error augmentation - rather than architecture golf, and every mission must state its expected gain before the Skeptic will let it through.

Metrics, Leakage and the Promotion Gate

All models report accuracy, precision, recall, F1 and ROC-AUC on the same frozen test split, plus a confusion matrix saved as an artifact. Splits are grouped by host star so the same star never appears on both sides of the gate - the leakage trap the Skeptic is explicitly briefed to hunt for. The promotion gate requires the challenger to beat the champion on F1 while ROC-AUC does not degrade, on the same split, with the run fully replayable; otherwise the attempt is recorded as a negative result together with the Evaluator's diagnosis. One further nuance keeps the science honest: because CANDIDATE labels are themselves uncertain, the primary demo metric treats CONFIRMED versus FALSE POSITIVE as ground truth and reports the candidate-inclusive numbers separately.

Backend Engineering (FastAPI)

The backend is the spine of the system: it hosts the REST API, the WebSocket event bus, the persistence layer and the experiment registry, and it spawns the ML sandbox. It is a single FastAPI application with a strict internal layout, so Member B can own it end to end while Member A plugs the agents in behind a thin interface. The layout below is the contract; any file that does not fit it gets discussed before it is created.

Backend layout

```
backend/
  app/
    main.py            # FastAPI factory, CORS, startup hooks
    routers/           # runs.py  experiments.py  candidates.py  data.py  ws.py
    services/          # orchestrator.py  llm.py  registry.py  graph.py
    db/                # models.py (SQLModel)  session.py
    schemas/           # pydantic contracts: AgentMessage, Proposal, Verdict
  ml_sandbox/
    train.py           # CLI: --config run.json --out artifacts/
    preprocess.py      # detrend, fold, bin - pure functions
    models/            # baseline.py  cnn.py
    data/              # cached .npz vectors + KOI parquet
  tests/               # schema tests, gate tests, replay tests
```

Method	Path	Purpose
POST	/api/missions	Start a research mission (question, round budget)
GET	/api/missions/{id}	Mission status: current round, agents idle or busy
GET	/api/experiments	Registry listing with metrics and verdicts
GET	/api/experiments/{id}/artifacts	Metrics JSON, confusion matrix, model weights
POST	/api/data/cache	Trigger KOI and light-curve caching (progress via WS)
GET	/api/graph	Nodes and edges for the knowledge-graph panel
WS	/ws/{mission_id}	Agent messages, run progress, metric updates

Table 7: REST and WebSocket surface of the backend.

WS event	Payload	Consumer
agent_message	agent, round, type, payload	Debate panel
run_progress	run_id, stage, percent	Status rail
run_metrics	run_id, metrics dictionary	Leaderboard
verdict	run_id, kept, rationale	Debate panel + leaderboard
system	level, text	Toast log

Table 8: WebSocket event contract - the single writer is the backend.

Data Model and the Registry

Table	Key fields	Purpose
missions	id, question, created_at, budget_rounds	One research objective
hypotheses	id, mission_id, config_json, proposer, status	Proposals and their fate
experiments	id, hyp_id, config_hash, seed, commit, metrics_json, artifact_path, verdict	The immutable registry
papers	id, title, abstract, source	The citation corpus
graph_edges	src_type, src_id, rel, dst_type, dst_id	The lightweight knowledge graph
messages	id, mission_id, agent, round, type, payload_json	The full debate transcript

Table 9: SQLite schema - six tables carry the entire system.

The registry is the self-improvement mechanism in miniature: experiments are append-only, config hashes make duplicates detectable, and the champion pointer is simply a query for the highest experiment that passed the gate. Promotion logic lives in one function, shown below, and is unit-tested with synthetic metric pairs so that nobody can argue with the gate at two in the morning. If the champion pointer is the heart of self-improvement, the append-only experiments table is its long-term memory.

Promotion gate (services/registry.py)

```
def promote(champion, challenger, margin=0.0):
    if challenger is None or not challenger.replayable:
        return Verdict.REJECT, "run not replayable"
    better_f1  = challenger.f1 > champion.f1 + margin
    safe_auc   = challenger.roc_auc >= champion.roc_auc - 1e-9
    if better_f1 and safe_auc:
        return Verdict.KEEP, "beats champion on F1, AUC not degraded"
    return Verdict.REJECT, diagnose(champion, challenger)
```

Continuous Integration with GitHub Actions

The GitHub repository runs the same discipline on every push and pull request: lint (ruff plus a black check, and eslint on the frontend), test (pytest with canned transcripts and promotion-gate tests), and a smoke job that starts the application, posts a mock mission with a stub LLM, and asserts that a verdict arrives within thirty seconds. The workflow lives in .github/workflows/ci.yml and runs entirely on GitHub Actions free-tier runners, with an opt-in progress-guard job that fails the build if PROGRESS.md goes stale for six hours during the sprint - so the team is never blocked by quota during the critical final hours.

Frontend: React Mission Control

The frontend is a single-screen mission control: the debate panel front and center, flanked by the light curve viewer and the experiment leaderboard, with a thin status rail along the bottom. The visual identity is a dark space theme with one accent color per agent, so the debate reads like a conversation between six clearly identified characters rather than a log dump. Everything the backend broadcasts appears within a second, and nothing on the demo screen ever requires a page refresh.

Panel	Purpose	Data source	Demo-critical
Debate panel	Streaming agent messages with avatars and round badges	WS agent_message	Yes
Light curve viewer	Folded flux with transit markers, before and after overlay	REST + WS	Yes
Leaderboard	Champion vs challenger table, keep and reject history	REST + verdict events	Yes
Mission input	Question box plus one-click mission templates	POST /api/missions	Yes
Graph panel	Entity graph: papers, hypotheses, runs	GET /api/graph	Stretch
Status rail	Run progress, toasts, LLM provider badge	WS system events	Nice

Table 10: The six panels, ranked by demo criticality.

Component and State Plan

Frontend tree (src/)

```
src/
  App.tsx                  # layout grid, dark theme
  api/useMissionSocket.ts  # WS hook: connect, dispatch by event type
  store/useOrbitStore.ts   # Zustand: messages[], metrics[], champion
  panels/
    DebatePanel.tsx        # one MessageBubble per agent type
    LightCurve.tsx         # Plotly: flux + transit shading
    Leaderboard.tsx        # sortable table, KEEP / REJECT chips
    MissionInput.tsx       # question box + templates
  components/              # Avatar, MetricChip, RoundBadge
```

State lives in a single Zustand store with one slice per event type, and the socket hook is the only writer, which makes the data flow trivially debuggable under pressure. Plotly renders the light curves because zoom, hover and annotations come for free. If the WebSocket reconnects mid-demo, the hook replays the

mission transcript from the REST API so the panel restores itself - a small feature that has a habit of saving live demos.

Polish is budgeted, not vibes-driven: agent avatars rendered as colored initials rather than emoji, a typing indicator while the LLM streams, keep and reject verdict chips in the palette's green and red, and a round banner whenever the Skeptic forces a revision. The stretch goal, attempted only after the end-to-end pass is green, is the react-force-graph panel - visually impressive and cheap, because the /api/graph endpoint already exists by then.

Tech Stack and Justification

The stack is boring on purpose: every choice is either an industry standard, already known to the team, or removable in ten minutes if it misbehaves. The table below is the contract - if a component is not on it, it does not enter the repository during the hackathon. That rule exists because the biggest killer of weekend projects is not hard problems; it is interesting side-quests.

Layer	Choice	Why (weekend-justified)	Alternative rejected
Frontend	React 18 + Vite + TypeScript	Instant HMR; the team already knows it	Next.js - SSR is useless on a LAN demo
Charts	Plotly.js	Zoom, hover and annotations built in	Hand-rolled D3 - a time sink
State	Zustand	About 30 lines, zero boilerplate	Redux - ceremony without payoff here
Backend	FastAPI + Uvicorn	Native WebSocket and pydantic contracts	Flask - weak async WS story
Database	SQLite + SQLModel	Zero config, typed models, one file to back up	Postgres - operations overhead
Graph	Edge table + NetworkX	In-memory, exportable as JSON	Neo4j - a second server to keep alive
LLM access	OpenAI-compatible base URL	One env var swaps any provider	Vendor SDK lock-in
LLM default	Gemini 2.5 Flash or GPT-4o-mini	Free or cheap tier; JSON mode is reliable	Local Ollama - needs a GPU we do not have
ML	scikit-learn + PyTorch (CPU)	Baseline speed plus CNN headroom	XGBoost - fine, but not needed
Astronomy	lightkurve, astropy, wotan	Mission-standard tooling with caching	Raw FITS parsing - days of work

Layer	Choice	Why (weekend-justified)	Alternative rejected
Data	NASA Exoplanet Archive + MAST	Authoritative labels and flux	Kaggle mirrors - stale subsets
CI/CD	GitHub Actions (lint, test, smoke)	Zero-setup, free runners, guards every push	GitLab CI - a second platform to learn

Table 11: The complete stack contract with weekend-justified rationale.

The Provider-Agnostic LLM Layer

All agents talk to exactly one connector module that reads three environment variables - base URL, API key and model name - and speaks the OpenAI chat protocol with JSON mode enabled. MiroFish proved this pattern works across DashScope, OpenAI-compatible clouds and local Ollama, and ORBIT adopts it unchanged. The default is the cheapest reliable cloud model, with Ollama as the offline fallback if the venue network dies, and a recorded-transcript replay mode as the final fallback if both fail. Three providers are tested on Day 0 so the swap has actually been exercised, not merely designed.

.env - swap providers without touching code

```
LLM_BASE_URL=https://api.groq.com/openai/v1   # or gemini proxy / localhost:11434/v1
LLM_API_KEY=sk-...
LLM_MODEL=llama-3.1-8b-instant                # any OpenAI-compatible model
```

Repository Layout and the Handover System

A hackathon repository fails in a predictable way: three people hold unwritten context, one person becomes the build, and handover happens by whispering across the table. ORBIT prevents this with a documentation system that is part of the definition of done, designed so that both a fresh teammate and an AI coding agent can become productive from the repository alone. The files are small, mandatory, and updated at fixed moments rather than when there is time - because there never is time.

Repository tree (root)

```
orbit/
  README.md              # what it is + 90-second runbook
  AGENTS.md              # THE coordination file - read first, update always
  PROGRESS.md            # append-only, timestamped work log
  HANDOVER.md            # per-module handover template, filled at swap points
  docs/
    FRONTEND.md  BACKEND.md  ML.md  AGENTS_SYSTEM.md   # module briefs + checklists
  backend/  ml_sandbox/  frontend/     # code (layouts in chapters 5-6)
  data/                  # cached KOI parquet + light curves (git-ignored)
  .github/workflows/ci.yml   # CI: lint, test, smoke, progress-guard
```

The AGENTS.md Protocol

AGENTS.md is the repository's operating manual for humans and AI agents alike: current system state, who owns what, branch and commit conventions, the schema contracts, and the never-do list - do not edit frozen test splits, do not reformat the registry, do not bump dependencies mid-hackathon. It sits at the repository root because modern AI coding tools read it automatically and because teammates actually will too. The rule that makes it work is enforced socially and technically: every pull request must update the Current State section, and during the hackathon window a CI job fails if PROGRESS.md has received no new entry in the previous six hours.

PROGRESS.md and HANDOVER.md

PROGRESS.md is an append-only log with one block per work session, each carrying a timestamp, an owner, and three lines: done, next, blockers. HANDOVER.md is filled whenever a module changes hands, even mid-day, and answers four questions: what works, what is broken, what is next, and what would I do differently. Together they mean any teammate can disappear for six hours - or an AI agent session can crash - and the replacement can reconstruct full context in about five minutes flat.

PROGRESS.md entry format

```
## 2026-11-08 14:20 - Member C (ML)
DONE: cached 1,842 KOI light curves; fold+bin pipeline green; baseline F1 0.87
NEXT: CNN v1 training run; wire metrics endpoint
BLOCKED: none
NOTE: TESS downloads ~6x slower than Kepler - capped cache at Kepler only
```

The 3-Day Sprint Plan

The sprint is planned backwards from demo freeze at hour 27 of 30: everything after freeze is rehearsal and fixes only. Each day has one integration checkpoint with a strict exit criterion, and no track may skip a checkpoint to finish one more feature. Day 1 produces a walking skeleton - one agent, one endpoint, one panel and one metric, wired end to end. Day 2 completes the feature set. Day 3 is integration, hardening, polish and rehearsal, in that order, with polish cut first whenever it threatens rehearsal time.

Day 0 - The Night Before (2-3 hours)

- Accounts ready: GitHub repository created and collaborators invited; LLM API keys tested and placed in .env.example (never the real keys committed).
- Environment: Python 3.11 virtualenv, Node 20, and ruff + black + pytest pre-commit hooks installed by every member.

- Data head start: KOI label table downloaded and roughly 200 light curves cached so Day 1 is never blocked on bandwidth.
- Documentation skeletons created: AGENTS.md, PROGRESS.md, HANDOVER.md and the four docs/ briefs from the chapter 8 templates.
- Commit convention agreed: conventional commits, pull requests even for solo work - it trains the CI muscle early.

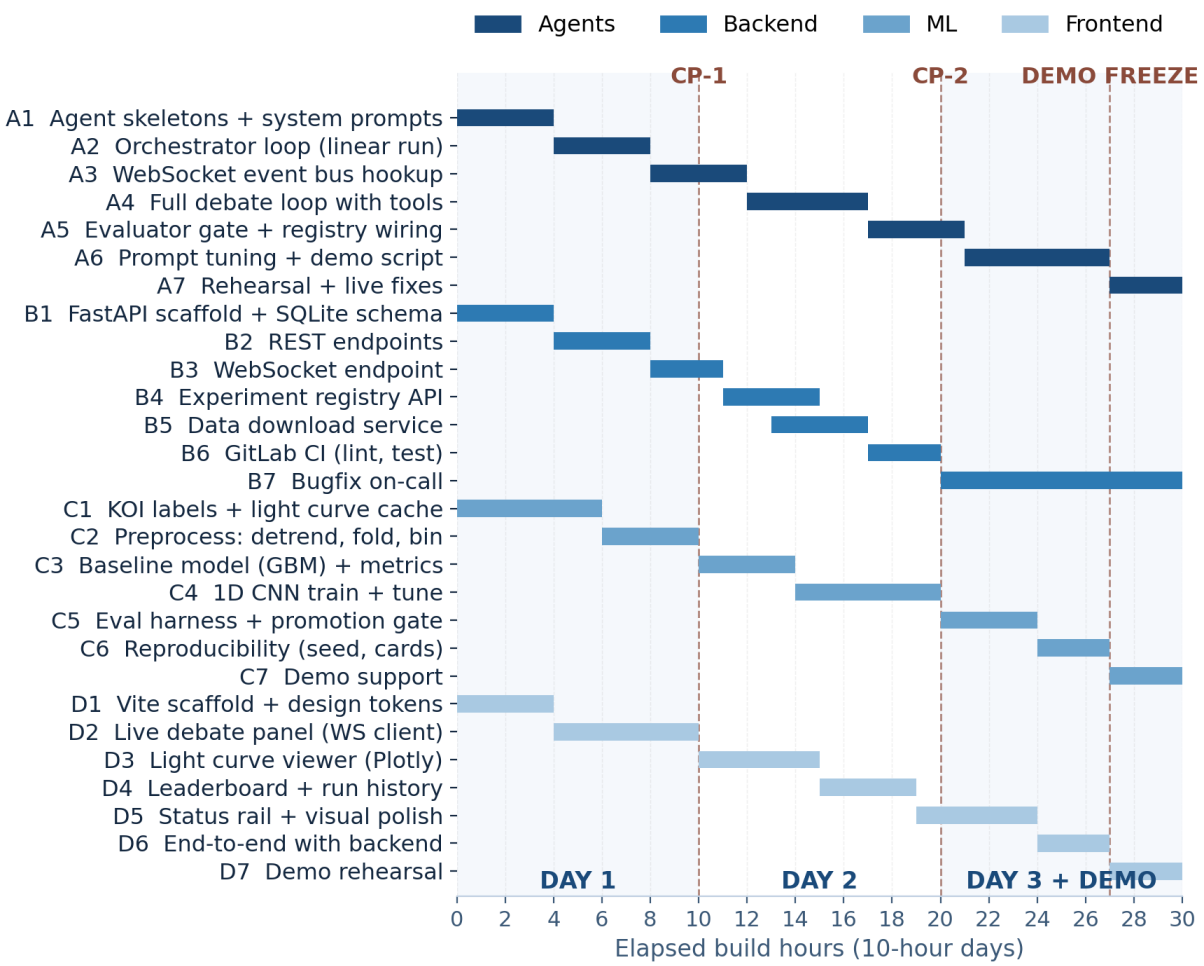


Figure 3: The 30-hour sprint across four tracks. CP-1 = walking skeleton, CP-2 = feature complete, freeze at hour 27.

Day 1 - Walking Skeleton (hours 0-10)

Hours	Focus	Exit criterion
0-4	B: FastAPI + SQLite schema. A: agent skeletons + prompts. C: data cache. D: Vite + design tokens	All four tracks compile and run
4-8	A: linear orchestrator. B: POST /missions + WebSocket. C: fold-and-bin vectors. D: chat panel shell	A mock mission round-trips
8-10	Integration checkpoint 1: mission in, one agent reply out, panel shows it; C: baseline trains	Walking skeleton demoable on one laptop

Table 12: Day 1 hour blocks and exit criteria.

Day 2 - Feature Complete (hours 10-20)

Hours	Focus	Exit criterion
10-14	A: full debate loop with tools. B: registry API. C: baseline + metrics. D: viewer + leaderboard	Registry rows written by real runs
14-17	A: Skeptic veto path. B: artifact store. C: CNN v1 training. D: verdict chips	CNN metrics visible in the leaderboard
17-20	Integration checkpoint 2: a full gated loop - propose, veto, train, verdict - recorded end to end	Feature complete

Table 13: Day 2 hour blocks and exit criteria.

Day 3 - Demo Hardening (hours 20-30)

Hours	Focus	Exit criterion
20-24	End-to-end bugfix; graph panel only if everything else is green; failure-path testing (kill the LLM mid-run)	No crash under injected faults
24-27	Visual polish, then demo freeze at hour 27: seed the database, record the backup video	Backup video exists on two devices
27-30	Two full rehearsals against the script; fix only demo-blocking bugs; pitch pass	Full 3-minute run completes under 3:30

Table 14: Day 3 hour blocks and exit criteria.

Freeze discipline is the plan's insurance policy. After hour 27 the team may only fix things that break the demo script itself; everything else goes into a post-hackathon list at the bottom of PROGRESS.md, which conveniently doubles as the roadmap section of the final submission form. Teams that respect a hard freeze consistently outperform teams that improve one more thing at 4 a.m.

Team Work-Split and Handover Protocol

Four members, four tracks, one integration lead. The split maps one person to one layer plus one shared artifact, so ownership questions never slow the build. The integration lead - Member A - owns the agent loop and the two contracts everyone else codes against: the WebSocket event schema and the experiment registry schema. Changing either contract requires the lead's sign-off in the pull request.

Member	Track	Owns	Key Day-1 deliverable	Key Day-2 deliverable
A	Agents + Integration	Orchestrator, prompts, event schema, checkpoints	Linear loop round-trip	Full gated loop
B	Backend / API	FastAPI, SQLite, registry, CI	Missions + WS endpoint	Registry + artifacts API
C	ML pipeline	Data, preprocessing, models, eval harness	Cached data + baseline F1	CNN trained + gate wired
D	Frontend	Panels, theme, WS client	Chat panel live	Viewer + leaderboard

Table 15: Ownership matrix - one owner per module, always.

With three members, Member B absorbs the CI pipeline and the data-download service into the backend track, Member C hands the frozen splits and evaluation harness to B at checkpoint 1, and Member D inherits the graph panel as an explicitly optional item. The role labels matter less than the rule underneath them: each module has exactly one owner at any moment, recorded in AGENTS.md rather than in anyone's memory.

The Handover Ritual

- At the end of each day and each checkpoint: a 15-minute stand-up around the Gantt; every member updates their PROGRESS.md block (done / next / blockers).
- AGENTS.md Current State is updated by the integration lead only - a single writer avoids merge fights.
- Any mid-session change of a module fills the four-question HANDOVER.md block before leaving the keyboard.
- Blockers are posted in the team chat with a reproducing command, never described from memory.
- Before any sleep longer than four hours on the final day: record a 60-second screen capture of your module working.

Demo Script and Pitch

The demo is three minutes and one take, so it is choreographed like a launch sequence. The single through-line: a real mission, watched live, ending in a measurable improvement the system keeps - and a rejection it remembers. Everything shown is either already in the database or produced live during the run; nothing on screen is typed faster than a paste.

Time	On screen	Spoken line (gist)
0:00-0:20	Mission control, dark theme, agents idle	ORBIT is a research team of six AI agents. Today's mission: find a better exoplanet detector.
0:20-0:50	Mission submitted; Researcher and Hypothesizer messages stream	It reads the literature and prior runs, then proposes a concrete change - and cites its evidence.
0:50-1:20	Skeptic critique lands; revision banner appears	Then it argues with itself. The Skeptic just flagged leakage risk - the proposal was revised before spending compute.
1:20-2:10	Experimenter progress, then metrics arrive; leaderboard updates	The experiment trains on real Kepler light curves. Challenger F1 0.91 against champion 0.87.
2:10-2:35	Evaluator verdict: KEPT; champion pointer moves; registry row shown	The Evaluator promotes it - and every number sits in an immutable registry you can replay.
2:35-3:00	Second mission: REJECTED verdict displayed	And when an idea fails, ORBIT remembers why. That is self-improvement with receipts.

Table 16: The three-minute demo choreography.

Backup Layers and Pitch Notes

Backups are layered, not improvised: first, a seeded database so the demo can skip the data phase entirely; second, a recorded 90-second screen capture of the full loop, cut at freeze and stored on two devices; third, if the LLM API is unreachable live, transcript-replay mode streams a cached debate while the presenter narrates over it. The pitch maps one sentence to each judging criterion: agents that challenge each other is the novel mechanism; evidence-gated self-improvement is measurable; persistent negative results are the honesty marker; and all of it runs on NASA's public data, which makes it real. The closing line stays humble and confident: ORBIT does not discover planets yet - it discovers better ways to look for them, and it can prove it.

Risk Register and Fallbacks

Every risk below has a likelihood, an impact, a mitigation and a fallback that does not depend on luck. The register is reviewed aloud at each integration checkpoint; a risk that fires converts immediately into its fallback rather than into a discussion. Likelihood and impact are graded low, medium or high relative to the 72-hour window.

Risk	L	I	Mitigation	Fallback
LLM API rate-limited or down	M	H	Batch agent calls; JSON mode pre-tested on two providers	Transcript-replay mode
Light-curve downloads slow	H	M	Day 0 head start; cap at Kepler; parallel fetch	300-target pre-scored subset
CNN training too slow on CPU	M	M	Subsample vectors; early stopping	GBM stays champion; CNN as stretch
WebSocket drops mid-demo	M	H	Auto-reconnect plus transcript replay	Refresh restores state from the DB
A teammate blocked over an hour	H	H	PROGRESS.md blockers + stand-ups	Integration lead reassigns the task
Scope creep: one more panel	H	M	Hard freeze at hour 27; stretch list only after CP-2	Graph panel is cut first
Schema drift between FE and BE	M	M	Pydantic contracts + CI schema tests	Typed fixtures as the shared truth
Leakage question stumps the team	L	H	Star-level split, unit-tested; Skeptic briefed	Show the split test code live

Table 17: The risk register with mitigations and pre-agreed fallbacks.

Three hard rules close the plan. First, mock mode is a feature: every external dependency - the LLM and MAST alike - has a stub, and the stubs are exercised daily. Second, the frozen test split is created once, on Day 1, and never regenerated; the promotion gate is only as credible as the split is stable. Third, at every checkpoint the question is not whether a module is finished but whether the next thirty minutes of the demo script work. If yes, continue; if no, the fallback for that segment ships instead of the dream version. ORBIT's promise to its judges is the same promise the system makes to itself: no claim without a replayable receipt.